



field guide · motion control · 6 projects · jun 2026

> make six things move on command_

Six EE Projects That Move With Precision

Steppers, servos, and brushless motors – six builds that turn electricity into precise, controlled motion. From a closed-loop position servo to a self-balancing robot to field-oriented control of a brushless motor. Cheap actuators; real control-systems engineering.

01	H-Bridge DC Motor Driver	Spin a motor either direction at any speed you choose – the front end of every robot	\$15
02	Closed-Loop Position Servo	Tell a motor to go to exactly 90 degrees and hold it there – feedback control, built from scratch	\$30
03	Self-Balancing Robot	A two-wheel robot that stands up and stays up – controlling an unstable system in real time	\$55
04	FOC Brushless Motor Drive	Drive a brushless motor smoothly and silently – the technology in drones, EVs, and robot dogs	\$45
05	2-Axis Pen Plotter	Two steppers that draw anything you give them – coordinated motion and interpolation	\$50
06	Haptic Smart Knob	A knob you can feel – render detents, springs, and end-stops in software, with a motor	\$40

● By Enes · @1nska · Electrical Engineer – a Build List field guide

H-Bridge DC Motor Driver

Spin a motor either direction at any speed you choose – the front end of every robot

\$15

beginner

1 weekend

why this matters

Before a robot can balance, drive, or steer, something has to turn a control signal into torque. An H-bridge does exactly that: four switches that push current through a DC motor in either direction, plus PWM that sets how hard. Build one, drive a motor forward and reverse, ramp its speed smoothly, and sense the current it draws, and you have the power stage that sits under every motor-control project in this guide. It is the cheapest, most fundamental motor build, and the payoff is immediate – a motor that does exactly what your code says.

WHAT THIS PROVES TO A RECRUITER

You understand H-bridge topology, PWM speed control, motor direction switching, freewheeling and why inductive loads need it, and current sensing – the power-stage foundation of motor-drive and robotics hardware.

→ **the job this maps to:** motor drives & robotics hardware – actuator, drone, and robotics companies, and power-electronics teams.

how it works

- ▶ **The H-bridge:** Four switches arranged in an H around the motor. Close one diagonal pair and current flows one way (forward); close the other pair and it reverses. Never close one side's top and bottom together – that is “shoot-through,” a dead short.
- ▶ **PWM sets the speed:** Rather than vary the voltage, you switch the bridge on and off thousands of times a second. The duty cycle (fraction on) sets the average voltage and thus the speed; the motor's inductance smooths the pulses into steady torque.
- ▶ **Freewheeling and flyback:** A motor is inductive, so when you switch it off the current must keep flowing somewhere. The bridge's body diodes (or added flyback diodes) give it a path, protecting the switches from voltage spikes – which is why an unprotected bridge dies fast.
- ▶ **Driving it from logic:** Two pins set direction and a PWM pin sets speed (or two PWM pins for sign-magnitude drive). A driver IC (DRV8871, TB6612, L298) packages the bridge and gate drive so you command it with logic levels.
- ▶ **Sensing current:** A small sense resistor (or the driver's current output) tells you how hard the motor is working. It is your window into torque, stalls, and faults – and the first ingredient of closed-loop control.

bill of materials

component	part / spec	qty	~cost
DC gear motor	6–12 V brushed	1	\$4
Motor driver	DRV8871 / TB6612 / L298N	1	\$3
Microcontroller	Arduino / ESP32	1	\$8
Sense resistor + passives	current sense, bulk caps	—	\$2
Power supply	battery or bench	1	—

build steps

1 Wire the driver and motor

Connect the motor to the driver outputs, logic pins to your microcontroller, and a motor supply separate from the USB rail. Add bulk capacitance across the motor supply. Confirm the motor twitches when you toggle a pin.

2 Drive forward and reverse

Set the direction pins and spin the motor each way at full duty. Confirm clean reversal and that nothing overheats; add a brief dead-time between direction changes.

3 Add PWM speed control

Apply PWM to the speed input and sweep the duty cycle. Watch the motor ramp from a crawl to full speed, and find the minimum duty where it starts to turn under load.

4 Add current sensing

Read the sense-resistor voltage (or driver current output) on the ADC. Stall the motor by hand briefly and watch the current spike — your torque and fault signal.

5 Wrap it in an API

Write `setMotor(speed, direction)` with limits and a soft-start ramp, and drive it from a potentiometer or serial. You now have a clean motor power stage to build closed-loop control on.

interview questions

- How does an H-bridge let a single supply drive a motor in both directions?
- What is shoot-through, and how do you prevent it?
- Why does PWM control motor speed, and what role does motor inductance play?
- Why must an H-bridge provide a freewheeling path for the motor current?
- How and why do you sense motor current?
- What is the difference between sign-magnitude and locked-antiphase PWM drive?

Closed-Loop Position Servo

Tell a motor to go to exactly 90 degrees and hold it there – feedback control, built from scratch

\$30

intermediate

1 weekend

why this matters

Open-loop, a motor just spins and you have no idea where it actually is. Add an encoder to measure its position and a PID controller to drive the error to zero, and the motor becomes a servo: command 90 degrees and it snaps there and holds, resisting if you push it. This is the single most important skill in motor control and robotics – closing the loop with feedback – and it is the foundation under every robot joint, CNC axis, and drone. Building your own PID position controller (rather than buying a servo) is the project that turns “I can spin a motor” into “I can control motion.”

WHAT THIS PROVES TO A RECRUITER

You understand encoders and quadrature decoding, feedback control, what each of the P, I, and D terms does, tuning, and steady-state error – the core of control-systems and robotics engineering, and the most transferable skill in the whole guide.

→ **the job this maps to:** control systems, robotics, and mechatronics engineering – robotics, automation, aerospace, and motion-control companies.

how it works

- ▶ **Measuring position:** A quadrature encoder outputs two pulse trains 90° out of phase; counting their edges and their order gives position and direction to a fraction of a degree. The microcontroller keeps a running count – the motor's measured angle.
- ▶ **The control loop:** Each cycle you compute $\text{error} = \text{target} - \text{measured}$, turn that error into a motor command, apply it, and repeat hundreds to thousands of times a second. The faster and more regular the loop, the tighter the control.
- ▶ **Proportional, integral, derivative:** P pushes proportionally to the error (the main effort). I accumulates error to kill the steady-state offset so it actually reaches the target. D responds to the rate of change to damp overshoot. Their sum is the motor command.
- ▶ **Tuning:** Too much P oscillates; too little is sluggish. I removes droop but can wind up; D smooths but amplifies noise. Tuning is finding the balance – and feeling, hands-on, what each term does is the real lesson.
- ▶ **Position, velocity, torque:** The same loop structure controls velocity (regulate speed) or, with current sensing, torque. Cascading them (position → velocity → current) is exactly how industrial servo drives are built.

BUILD NOTE

Reach for a microcontroller with a fast, regular loop and a real quadrature input. Decode the encoder cleanly first and confirm the count tracks the shaft before you close any loop – a flaky encoder makes PID impossible to tune.

key design decisions

- **Why closed-loop over a stepper's open-loop precision?** A stepper holds position without feedback but can silently lose steps under load and wastes power holding still. A closed-loop servo knows its true position, recovers from disturbances, and only works as hard as needed – which is why robots and CNC increasingly use closed-loop drives.
- **Why does the loop rate matter so much?** Control assumes the world barely changes between updates. A slow loop reacts late, so it must use gentle gains or it oscillates; a fast, regular loop can use higher gains for tighter, stiffer control. Jitter in the loop timing is as harmful as a slow loop.
- **Why add the integral term if P almost works?** Proportional control alone leaves a residual error – the motor settles where P's push balances friction and load, short of the target. The integral term accumulates that leftover error until it is driven to zero, so the servo actually reaches the commanded angle and holds it under load.

bill of materials

component	part / spec	qty	~cost
DC gear motor with encoder	6-12 V + quadrature encoder	1	\$12
Motor driver	DRV8871 / TB6612	1	\$4
Microcontroller	ESP32 / Arduino (fast loop)	1	\$8
Knob / input	potentiometer for target	1	\$2
Passives + breadboard	wiring	—	\$4

build steps

1 Read the encoder

Wire the quadrature encoder and decode it (interrupts or a hardware quadrature peripheral). Turn the shaft by hand and confirm the count rises and falls correctly with direction.

2 Close a P loop

Compute $\text{error} = \text{target} - \text{count}$ and drive the motor with a command proportional to it. Raise the gain until the motor moves to the target – and watch it overshoot or buzz. You just closed the loop.

3 Add D, then I

Add a derivative term to damp the overshoot, then an integral term to remove the residual offset so it lands exactly on target. Observe each term's effect as you add it.

4 Tune and test

Step the target ($0 \rightarrow 90^\circ$), watch the response, and tune P / I / D for a fast, clean settle with minimal overshoot. Push the shaft and feel it resist and return.

5 Make it a servo API

Wrap it as `moveTo(angle)` with a fixed loop rate, limits, and a potentiometer or serial target. You now have a from-scratch closed-loop servo – the heart of every later robotics project.

interview questions

- How does a quadrature encoder encode both position and direction?
- What does each of the P, I, and D terms contribute to a controller?
- Why does proportional-only control leave a steady-state error?
- Why does control-loop rate (and its consistency) matter for stability?
- How would you extend a position controller to control velocity or torque?
- What is integrator windup, and how do you prevent it?

Self-Balancing Robot

A two-wheel robot that stands up and stays up – controlling an unstable system in real time

\$55

advanced

1–2 weekends

why this matters

A two-wheel balancing robot is an inverted pendulum: left alone it falls, so it only stays upright because a controller is constantly catching it. An IMU measures the tilt, a control loop computes how to drive the wheels to stay under the centre of mass, and the robot balances – leaning into a push and recovering. It is the most visually striking build in this guide and a genuine lesson in controlling an unstable system, sensor fusion, and cascaded control. If the closed-loop servo taught you to hold a position, this teaches you to stabilise something that actively wants to fall.

WHAT THIS PROVES TO A RECRUITER

You understand inverted-pendulum dynamics, IMU sensor fusion (accelerometer + gyro, complementary or Kalman filtering), cascaded PID (angle and speed loops), and real-time control of an unstable plant – exactly the hard part of mobile-robotics and drone control.

→ **the job this maps to:** robotics and control engineering – mobile-robot, drone, and balancing / legged-robot companies, and controls teams.

how it works

- ▶ **The inverted pendulum:** A body balanced above its wheels is unstable – any lean grows unless corrected. The controller drives the wheels in the direction of the fall so the pivot stays under the centre of mass, the same way you balance a broom on your hand.
- ▶ **Measuring tilt:** An accelerometer gives a noisy absolute tilt; a gyro gives a clean tilt rate but drifts. Fusing them (a complementary or Kalman filter) yields a stable, fast tilt angle – the single most important signal in the system.
- ▶ **The balance loop:** A PID controller takes the tilt error and commands wheel torque to correct it. Get this loop right and the robot stands; it leans into a push and drives forward to recover, which looks like magic but is just feedback.
- ▶ **Cascaded control:** A second, slower loop controls wheel speed and position by gently biasing the target tilt – so the robot not only balances but holds its ground (or drives where commanded) instead of slowly rolling away.
- ▶ **Why timing and tuning are everything:** An unstable plant punishes slow or jittery loops instantly. A fast, regular loop and careful tuning are the difference between standing and thrashing – the core discipline of real-time control.

BUILD NOTE

Don't fight three problems at once. Use a proven two-wheel chassis, geared motors with encoders, and an MPU6050; get a clean tilt angle first, then add the balance loop, then the speed loop. Tune one loop at a time.

bill of materials

component	part / spec	qty	~cost
Geared motors + encoders (×2)	6-12 V with quadrature	2	\$20
Motor driver	dual H-bridge (TB6612)	1	\$8
IMU	MPU6050 / MPU9250	1	\$3
Chassis + wheels	two-wheel balance frame	1	\$12
Microcontroller	ESP32	1	\$8
Battery	LiPo + holder	1	\$8

build steps

1 Build the platform

Assemble the two-wheel chassis with motors, encoders, driver, the IMU mounted near the wheel axis, and battery. Keep the centre of mass reasonable and the wiring tidy – mechanics matter here.

2 Get a clean tilt angle

Read the IMU and fuse accel + gyro into a stable tilt estimate with a complementary (or Kalman) filter. Tip the robot by hand and confirm the angle is fast and drift-free.

3 Close the balance loop

Feed tilt error into a PID that commands wheel torque. Hold the robot near upright and tune until it briefly balances on its own, then stands. Expect a lot of tuning.

4 Add the speed loop

Add a slower outer loop that nudges the target tilt to control wheel speed and position, so the robot holds station instead of drifting. Tune so it balances without rolling away.

5 Make it drive

Add commands (serial / Bluetooth) that bias the loops to move forward, back, and turn while staying balanced. Now push it and watch it catch itself.

interview questions

- Why is a balancing robot an example of an unstable system?
- Why fuse accelerometer and gyro data instead of using either alone?
- What is the role of the inner (angle) versus outer (speed) control loop?
- Why does an unstable plant demand a fast, low-jitter control loop?
- How does the robot recover from a push, in control terms?
- What is sensor fusion, and why is a complementary or Kalman filter used?

FOC Brushless Motor Drive

Drive a brushless motor smoothly and silently – the technology in drones, EVs, and robot dogs

\$45

advanced

1–2 weekends

why this matters

Brushless (BLDC / PMSM) motors are everywhere high performance matters – drones, EVs, robot joints, gimbals – because they are efficient, powerful, and long-lived. But they need electronic commutation: you must energise the right coils at the right instant based on rotor position. Field-Oriented Control (FOC) does this optimally, producing smooth, silent, full-torque-at-any-speed motion, even holding still. Building a FOC drive – with a position sensor, a three-phase driver, and a library like SimpleFOC – is the leap into modern motor control and one of the most in-demand EE skills in robotics and electrified everything.

WHAT THIS PROVES TO A RECRUITER

You understand three-phase brushless motors, electronic commutation, the idea of FOC (Clarke / Park transforms, controlling torque-producing current directly), rotor position sensing, and three-phase driver hardware – core to motor-drive, drone, EV, and robotics-actuator engineering.

→ **the job this maps to:** motor-drive, drone, EV, and robotics-actuator engineering – drone, electric-vehicle, and advanced-robotics companies.

how it works

- ▶ **Why brushless needs commutation:** A BLDC has no brushes to switch its coils, so the controller must energise the three phases in the right sequence based on where the rotor is. Get the timing wrong and it cogs, stalls, or runs hot; get it right and it is smooth and efficient.
- ▶ **Knowing the rotor angle:** FOC needs accurate rotor position – from a magnetic encoder (AS5600), Hall sensors, or sensorless back-EMF estimation. Position is to a brushless drive what the encoder was to your servo: without it, no smooth control.
- ▶ **Field-oriented control:** FOC mathematically rotates the three-phase currents into the rotor's frame (the Clarke and Park transforms) so you can directly command the torque-producing current and keep it optimally aligned with the rotor – maximum torque per amp, at any speed.
- ▶ **The three-phase driver:** Three half-bridges (six switches) drive the three motor phases with PWM. A gate driver and current sensing turn your FOC commands into the actual phase currents – the brushless equivalent of project 1's H-bridge, scaled up.
- ▶ **What you get:** Smooth motion down to zero speed, full torque while holding still (no cogging), high efficiency, and quiet operation – the qualities that make brushless drives the default for drones, EVs, gimbals, and robot joints.

BUILD NOTE

Don't write FOC from scratch first. Start with SimpleFOC on a small gimbal BLDC, an AS5600 magnetic encoder, and a three-phase driver board – get smooth open-loop motion, then closed-loop torque and position, then study what the transforms are doing underneath.

bill of materials

component	part / spec	qty	~cost
Gimbal BLDC motor	small, high pole-count	1	\$12
3-phase driver board	SimpleFOC Mini / DRV8313	1	\$14
Magnetic encoder	AS5600 (I2C / analog)	1	\$3
Microcontroller	ESP32 / STM32	1	\$8
Power + passives	supply, bulk caps	–	\$8

build steps

1 Wire motor, driver, sensor

Connect the BLDC to the three-phase driver, the AS5600 to read the rotor angle, and the driver control pins to your microcontroller. Confirm the encoder reports a clean absolute angle as you turn the shaft.

2 Open-loop spin

Run SimpleFOC in open-loop velocity mode to energise the phases and spin the motor smoothly. This proves the driver and phase wiring before you trust position feedback.

3 Calibrate the sensor

Run the alignment routine so the controller learns the rotor's electrical zero and direction relative to the encoder. This is essential for FOC to apply current in the right place.

4 Closed-loop torque and position

Switch to closed-loop modes – command a torque and feel the motor hold it, then command positions and watch it move smoothly and hold with no cogging, even at standstill.

5 Look under the hood

With it working, dig into the Clarke / Park transforms and current loops the library runs, and tune them. You now understand the drive behind drones, EVs, and robot joints.

interview questions

- Why does a brushless motor require electronic commutation?
- What problem does Field-Oriented Control solve, at a high level?
- Why does FOC need accurate rotor position, and how is it obtained?
- What do the Clarke and Park transforms accomplish in FOC?
- Why can a FOC drive produce full torque at zero speed without cogging?
- How does a three-phase inverter differ from a single H-bridge?

2-Axis Pen Plotter

Two steppers that draw anything you give them – coordinated motion and interpolation

\$50

intermediate

1-2 weekends

why this matters

A single controlled motor is one thing; making several move together to trace a precise path is another. A 2-axis pen plotter coordinates two stepper motors so a pen follows straight lines, curves, and shapes – the same coordinated-motion and interpolation problem at the heart of 3D printers, CNC machines, and laser cutters. You build the motion controller: parse a path, break it into synchronised X / Y steps with proper acceleration, and watch the machine draw. It is a deeply satisfying, visual build that teaches trajectory planning and multi-axis coordination, a different and essential facet of motor control.

WHAT THIS PROVES TO A RECRUITER

You understand stepper driving and microstepping, coordinating multiple axes, linear interpolation (Bresenham / DDA), acceleration profiles (trapezoidal motion), and turning a path into synchronised motor steps – core CNC, 3D-printing, and motion-control engineering.

→ the job this maps to: motion control, CNC, and robotics engineering – 3D-printer, CNC, and automation-equipment companies.

how it works

- ▶ **Stepper basics:** A stepper moves in precise increments; microstepping subdivides each step for smoothness and resolution. Driving it open-loop gives accurate positioning without an encoder – as long as you never exceed its torque, or it skips steps.
- ▶ **Coordinating two axes:** To draw a straight diagonal, both motors must move together at the right relative rates so the pen follows the line, not a staircase. Interpolation (Bresenham or a DDA) decides which axis steps when, keeping the path straight.
- ▶ **Acceleration profiles:** Steppers lose torque with speed and skip if jerked, so each move ramps up and down (a trapezoidal velocity profile). Smooth acceleration is what lets the machine move fast without losing position.
- ▶ **From path to motion:** A drawing is a list of line and curve segments. The controller converts each into coordinated, ramped X / Y step pulses – exactly the job a 3D printer or CNC controller does turning G-code into motion.
- ▶ **The pen axis:** A small servo or solenoid lifts and drops the pen between strokes, so the machine draws only where it should – the simplest possible “tool” on a motion platform.

BUILD NOTE

Don't reinvent the firmware stack. A CNC shield with stepper drivers (A4988 / DRV8825) plus GRBL gets you drawing fast; then study how it interpolates and ramps, or write your own simple line / arc interpolator to really learn it.

bill of materials

component	part / spec	qty	~cost
Stepper motors (×2)	NEMA-17	2	\$14
Stepper drivers	A4988 / DRV8825 (×2)	2	\$4
Controller	Arduino + CNC shield (or ESP32)	1	\$10
Frame, belts, pulleys	2-axis gantry / CoreXY	1	\$15
Pen-lift servo	SG90 + linkage	1	\$5

build steps

1 Build the gantry

Assemble a 2-axis frame (gantry or CoreXY) with the two steppers, belts, and a pen carriage. Square and tension it – slop here shows up directly in the drawing.

2 Drive each axis

Wire the stepper drivers, set their current limits, and jog each axis independently. Confirm correct direction and that a commanded distance matches the actual travel.

3 Interpolate a line

Implement (or run GRBL's) coordinated stepping so a diagonal move drives both axes together into a straight line. Draw a square and check the corners and edges.

4 Add acceleration

Add trapezoidal ramping so moves speed up and slow down smoothly. Push the speed and confirm it no longer skips steps or loses position.

5 Draw real artwork

Feed it paths (G-code or your own format), add pen up / down, and let it draw. Tune speed, acceleration, and current for clean lines – your own little CNC.

interview questions

- Why can a stepper be driven open-loop and still position accurately?
- What is microstepping, and what does it buy you?
- How do you coordinate two axes to draw a straight diagonal line?
- Why do stepper moves need acceleration profiles?
- What causes a stepper to lose steps, and how do you avoid it?
- How does a CNC or 3D-printer controller turn a path into motor motion?

Haptic Smart Knob

A knob you can feel – render detents, springs, and end-stops in software, with a motor

\$40

advanced

1–2 weekends

why this matters

Most knobs are passive; this one fights back. By driving a brushless motor with FOC and reading its exact angle, you can render torque as a function of position – so the same physical knob can feel like clicky detents, a smooth spring that returns to centre, or a hard end-stop, all changed in software. It is the technology behind high-end car controls and the viral “smart knob,” and it is a brilliant lesson in torque control and haptics: you are programming a force field you can feel with your fingers. It builds directly on FOC and turns motor control into a user interface.

WHAT THIS PROVES TO A RECRUITER

You understand torque (current) control of a brushless motor, reading precise position, and rendering position-dependent forces (virtual springs, detents, walls) – the basis of haptics and force-feedback systems. It shows you can use a motor as an output device, not just an actuator.

→ the job this maps to: haptics, human-machine interface, and robotics engineering – automotive HMI, consumer-device, and force-feedback / robotics teams.

how it works

- ▶ **Torque from current:** In FOC torque mode you command the motor's torque directly via its current. Instead of asking the motor to reach a position, you ask it to push with a certain force – the key that makes haptics possible.
- ▶ **Position-dependent force:** Read the precise angle and compute the torque to apply as a function of it. A torque that pulls toward the nearest of N evenly spaced angles feels like detents; a torque proportional to distance from centre feels like a spring.
- ▶ **Detents, springs, walls:** Different force-versus-angle functions give different feels – clicky notches, a smooth centring spring, a free spin with hard end-stops, or a viscous “thick oil” drag (torque opposing velocity). All are just math on the angle and speed.
- ▶ **The control loop:** A fast loop reads angle (and speed), evaluates the feel function, and commands the corresponding torque – hundreds to thousands of times a second, so the force feels instantaneous and stable under your fingers.
- ▶ **A motor as an output device:** This flips motor control on its head: the motor is now a display for forces, not a mover of loads. The same idea scales to force-feedback joysticks, steering wheels, and robot end-effectors that feel their environment.

BUILD NOTE

This is FOC plus a feel function. Reuse the project-4 setup (gimbal BLDC, AS5600, three-phase driver, SimpleFOC in torque mode); the new work is mapping angle to commanded torque to create each “feel.”

bill of materials

component	part / spec	qty	~cost
Gimbal BLDC motor	small, high pole-count	1	\$12
3-phase driver board	SimpleFOC Mini / DRV8313	1	\$14
Magnetic encoder	AS5600	1	\$3
Microcontroller	ESP32 / STM32	1	\$8
Knob + power	cap, supply, knob cap	–	\$3

build steps

1 Reuse the FOC drive

Start from a working FOC setup (project 4) in torque mode with the AS5600 reading angle. Confirm you can command a steady torque and feel the motor push.

2 Render a spring

Apply a torque proportional to the negative of the angle from a centre point. Let go of the knob and feel it spring back to centre; change the gain to make it stiff or soft.

3 Render detents

Compute the nearest of N evenly spaced positions and apply a torque pulling toward it. Turn the knob and feel discrete clicks; change N to change the number of detents.

4 Add walls and damping

Add hard end-stops (a large restoring torque past a limit) and a velocity-proportional drag for a “thick” feel. Switch between feels in software on the same hardware.

5 Make it a control

Map knob position or detents to an action (volume, menu, brightness) over serial, so your programmable haptic knob actually controls something. One knob, any feel.

interview questions

- What is torque (current) control, and why is it the basis of haptics?
- How do you make a motor render a virtual spring?
- How would you create the feel of detents in software?
- Why does the haptic control loop need to run fast?
- What does it mean to use a motor as an output device rather than an actuator?
- How would you add a velocity-dependent (viscous) feel?

> wrap

A motor is just electricity – control turns it into a robot.

An H-bridge, a PID servo, a balancing robot, a FOC brushless drive, a coordinated plotter, a haptic knob – six steps from “spin a motor” to “command motion precisely.” The actuators are cheap (a few dollars of motor, a driver board, an encoder), and the value is entirely in the control: the feedback, the tuning, the commutation, the trajectory. That is exactly what robotics, motion-control, and electrified-everything teams hire for.

// the skill ladder

Build them in order and each one hands you the technique the next one needs:

open-loop drive → closed-loop PID → balancing → field-oriented control → coordinated motion → haptics

! BEFORE YOU SPIN IT UP

Motors mean moving parts, real current, and LiPo batteries. Set the driver’s current limit, keep fingers and cables clear of wheels and gears, fuse or current-limit your supply, and don’t bench-test a balancing robot near your face. Power down before you rewire.

DO THIS

Build even one. Film it – a motor snapping to a target and resisting your push, a robot catching itself, a brushless motor holding torque in silence, a machine drawing – and put the code on GitHub with the control loop, your tuning, and an honest note on what works and what doesn’t. One motion demo and one clean repo prove you can make hardware do exactly what you tell it.

Full build notes, parts links, and the next field guide go out in **The Build List**. The people actually shipping these – and trading PID gains and FOC tricks – are in **Ohm’s Club**.

– Enes (@1nska) · Electrical Engineer